

Kothon Frontend — Teacher Notes

এই document এ Kothon chat app এর frontend এ কী কী করা হয়েছে, কোনটার কাজ কী, কেন করা হয়েছে — সব লেখা আছে।

Folder Structure

```
frontend/src/
├── components/
│   ├── layout/
│   │   └── ProtectedRoute.jsx    ← login check
│   ├── Sidebar.jsx              ← chat list (left side)
│   └── ChatWindow.jsx           ← messages (right side)
├── context/
│   ├── AuthContext.jsx          ← user login info সবখানে share
│   └── SocketContext.jsx        ← socket connection সবখানে share
├── pages/
│   ├── HomePage.jsx             ← main chat page
│   ├── LoginPage.jsx            ← login form
│   └── RegisterPage.jsx         ← register form
├── services/
│   └── api.js                   ← সব API calls এখানে
├── App.jsx                      ← routes define
├── main.jsx                     ← entry point
└── index.css                   ← styles
```

Context — কী আর কেন?

Context কী?

React এ data parent → child এভাবে pass হয় (props)। কিন্তু অনেক গভীরে pass করতে হলে বারবার props দিতে হয়। Context দিয়ে যেকোনো **component** থেকে data access করা যায়।

AuthContext.jsx — কী করে?

User login info সব component এ share করে।

যা আছে:

- `user` — logged in user এর info (username, email, id)
- `loading` — app load হচ্ছে কিনা
- `login(loginInput, password)` — login function
- `register(username, email, password)` — register function
- `logout()` — logout function

কেন লাগে:

- Sidebar এ user এর নাম দেখাতে
- ChatWindow এ "আমার message" কিনা check করতে
- ProtectedRoute এ login আছে কিনা check করতে

কীভাবে use করি:

```
jsx
const { user, login, logout } = useAuth();
```

কীভাবে কাজ করে:

```
App load → localStorage এ token আছে?
├─ হ্যাঁ → /api/auth/me call → user set হলো
└─ না → loading false, login page দেখাবে
```

SocketContext.jsx — কী করে?

```
Socket.io connection সব component এ share করে।
```

যা আছে:

- `socket` — socket.io connection object

কেন লাগে:

- ChatWindow এ message send/receive করতে
- Sidebar এ নতুন message listen করতে

কীভাবে use করি:

```
jsx
```

```
const { socket } = useSocket();
```

কীভাবে কাজ করে:

```
User login করলো → user state set হলো  
→ useEffect run হলো (dependency: [user])  
→ localStorage থেকে token নিলো  
→ socket.io connect করলো token দিয়ে  
→ Backend socket middleware JWT verify করলো  
→ Connection successful!
```

```
User logout করলো → user null হলো  
→ useEffect cleanup চললো  
→ socket disconnect হলো
```

Important: `useEffect` এ `return` মানে cleanup — user change হলে আগের socket disconnect হয়, নতুন connect হয়।

Pages

LoginPage.jsx

কাজ: User login form

Flow:

```
User email/mobile + password দেয়  
→ form submit হয়  
→ AuthContext এর login() call হয়  
→ API call: POST /api/auth/login { loginInput, password }  
→ token localStorage এ save হয়  
→ user state set হয়  
→ ProtectedRoute redirect করে HomePage এ
```

শেখার বিষয়:

- `useState` — form data রাখতে
- `handleChange` — input change হলে state update
- `handleSubmit` — form submit হলে API call
- `try/catch` — error handling

- `loginInput` নাম দিলাম কারণ email বা mobile দুটোই দেওয়া যায়

RegisterPage.jsx

কাজ: নতুন user register

Flow: LoginPage এর মতোই, শুধু API endpoint আলাদা।

HomePage.jsx

কাজ: Main chat page — Sidebar + ChatWindow

যা আছে:

- `selectedRoom` state — কোন chat select করা আছে
- Sidebar কে `onRoomSelect` function pass করে
- ChatWindow কে `room` আর `onBack` pass করে

Responsive কীভাবে:

```
Mobile (< 768px):
└─ selectedRoom নেই → Sidebar fullscreen
└─ selectedRoom আছে → ChatWindow fullscreen + back button

Desktop (>= 768px):
└─ Sidebar + ChatWindow পাশাপাশি
```

Tailwind responsive classes:

- `md:` prefix = 768px+ (tablet/desktop)
- `hidden md:flex` = mobile এ লুকানো, desktop এ দেখাবে
- `md:hidden` = desktop এ লুকানো, mobile এ দেখাবে

Components

Sidebar.jsx

কাজ: Chat list দেখানো

Props পায়:

- `onRoomSelect` — chat click করলে HomePage এ `selectedRoom` set হয়
- `selectedRoom` — কোন chat selected আছে (highlight করতে)

State:

- `rooms` — সব chat rooms এর list

কীভাবে data আনে:

```
Component load হলো (useEffect, dependency: [])  
→ roomAPI.getMyChats() call  
→ response থেকে rooms set করলো  
→ UI render হলো
```

Socket listener — কেন?

```
নতুন message আসলে sidebar এ last message update হয়।  
socket.on('newMessage') → ওই room এর lastMessage replace করে।
```

Important: `socket.off('newMessage', handleNew)` — specific function remove করে, ChatWindow এর listener মুছে যায় না!

Chat item click করলে:

```
onClick={() => onRoomSelect(room)}  
→ HomePage এ selectedRoom set হয়  
→ ChatWindow re-render হয় নতুন room দিয়ে
```

Other member বের করা:

```
jsx  
  
const otherMember = room.members.find(m => m.user._id !== user.id);
```

নিজে বাদে অন্যজনের নাম দেখানোর জন্য।

ChatWindow.jsx

কাজ: Messages দেখানো, message পাঠানো

Props পায়:

- `room` — কোন chat room open
- `onBack` — back button click করলে selectedRoom null হয়

State:

- `messages` — এই room এর সব messages
- `content` — input box এ কী লেখা আছে

Ref:

- `messagesEndRef` — auto scroll করতে

Socket events:

```
Room join:
useEffect চালু হলো (dependency: [room, socket])
→ socket.emit('joinRoom', room._id)
→ Backend: MongoDB থেকে messages আনে
→ socket.emit('loadMessages', messages) পাঠায়
→ Frontend: socket.on('loadMessages') → setMessages
```

```
নতুন message আসলে:
→ socket.on('newMessage') → messages array তে add হয়
→ pending (temp) messages filter হয়
```

```
Room ছেড়ে যাওয়া (cleanup):
→ socket.emit('leaveRoom', room._id)
→ socket.off('loadMessages', handleLoad)
→ socket.off('newMessage', handleNew)
```

Message পাঠানো (Optimistic Update):

```
User type করে Enter চাপে / Send চাপে
→ sendMessage() call হয়
→ temp message তৈরি হয় (pending: true)
→ UI তে instantly দেখায় (optimistic)
→ socket.emit('sendMessage', data) — server এ পাঠায়
→ Server save করে → newMessage emit করে
→ Frontend: pending message filter হয়, real message আসে
```

কেন optimistic? Server round trip এ 1-2 sec লাগে — user কে wait করাতে চাই না।

Auto scroll:

```
messages state change হলে (useEffect, dependency: [messages])
→ messagesEndRef.current.scrollToView({ behavior: 'smooth' })
→ নিচে scroll হয়
```

chatName logic:

```
Group chat → room.name দেখাবে  
Private chat → otherMember এর nickname (থাকলে) বা username
```

isMe check:

```
jsx  
  
const isMe = msg.sender._id === user.id;
```

আমার message হলে ডানে green bubble, অন্যের হলে বামে white bubble।

ProtectedRoute.jsx

কাজ: Login ছাড়া page দেখতে দেবে না

কীভাবে কাজ করে:

```
ProtectedRoute:  
├─ loading → "Loading..." দেখায়  
├─ user নেই → /login এ redirect  
└─ user আছে → children render করে  
  
PublicRoute (login/register page):  
├─ loading → "Loading..." দেখায়  
├─ user আছে → / (home) এ redirect  
└─ user নেই → children render করে
```

services/api.js

কাজ: সব API calls এক জায়গায়

Axios instance:

```
baseUrl: http://localhost:5000/api  
timeout: 10 seconds
```

Request interceptor:

```
প্রতিটা request এ localStorage থেকে token নিয়ে  
Authorization: Bearer <token> header এ attach করে
```

Response interceptor:

```
401 (Unauthorized) আসলে:  
→ token মুছে দেয়  
→ /login এ redirect করে
```

API functions:

```
authAPI.login({ loginInput, password }) → POST /api/auth/login  
authAPI.register({ ... }) → POST /api/auth/register  
authAPI.getMe() → GET /api/auth/me  
roomAPI.getMyChats() → GET /api/room/getMyChats  
roomAPI.getOrCreate(data) → POST /api/room/createOrLoad  
userAPI.search(q) → GET /api/users/search?q=...
```

CSS / Styling Notes

Tailwind CSS কীভাবে কাজ করে: HTML class এ directly style দেওয়া যায়। আলাদা CSS file লাগে না।

যেগুলো বেশি use হয়েছে:

```
flex → flexbox layout  
flex-col → উপর থেকে নিচে  
items-center → vertically center  
justify-between → দুই প্রান্তে  
gap-3 → elements এর মাঝে gap  
p-4 → padding সব দিকে  
px-3 py-2 → horizontal/vertical padding আলাদা  
w-10 h-10 → width/height  
rounded-full → গোল  
bg-[#202c33] → custom color  
text-sm → small text  
overflow-y-auto → vertical scroll  
cursor-pointer → click করা যায় বোঝাতে  
hover:bg-[#202c33] → hover করলে color change  
transition-colors → smooth color transition
```

Responsive:


```
w-full md:w-80      → mobile: full width, desktop: 320px
hidden md:flex      → mobile: hidden, desktop: show
md:hidden           → mobile: show, desktop: hidden
```

Message animation (index.css):

CSS

```
@keyframes slideUp {
  from { opacity: 0; transform: translateY(20px); }
  to   { opacity: 1; transform: translateY(0); }
}
.message-new { animation: slideUp 0.3s ease-out; }
```

Data Flow Summary

Login:

```
LoginPage → AuthContext.login() → API → token save → user set
→ SocketContext → socket connect → ready!
```

Chat list load:

```
Sidebar useEffect → roomAPI.getMyChats() → rooms set → render
```

Chat select:

```
Sidebar click → onRoomSelect(room) → HomePage selectedRoom set
→ ChatWindow re-render → joinRoom emit → loadMessages receive → render
```

Message send:

```
Input → sendMessage() → temp message (instant UI) → socket.emit
→ Backend save → newMessage emit → ChatWindow receive → render
→ Sidebar receive → lastMessage update
```

Message receive:

```
Other user sends → Backend emit → socket.on('newMessage')
→ ChatWindow: messages update → render
→ Sidebar: lastMessage update → render
```

⚠ Common Mistakes & Fixes

ভুল	কারণ	Fix
Import path wrong	<code>./</code> vs <code>../</code> confusion	File কোথায় আছে দেখো, কত folder উপরে যেতে হবে count করো
useState outside function	React rule — hooks function এর ভেতরে	সব useState/useEffect function এর শুরুতে
useEffect nested	useEffect এর ভেতরে useEffect রাখা যায় না	আলাদা আলাদা useEffect লেখো
socket.off সব listener মুছে দেয়	<code>socket.off('event')</code> সব listener remove করে	<code>socket.off('event', specificFunction)</code> use করো
Duplicate import	same জিনিস দুইবার import	একবারই import করো
props receive না করা	function এ <code>{ propName }</code> না লেখা	<code>function Component({ prop1, prop2 })</code>

🚀 এখনো বাকি আছে

- ☐ Search functionality — user search করে নতুন chat শুরু
- ☐ Typing indicator — "karim is typing..."
- ☐ Online status — real-time online/offline
- ☐ Read receipts — blue tick
- ☐ Group chat UI
- ☐ Image/file sharing
- ☐ Profile page
- ☐ Settings page
- ☐ Register page fix (mobileNo add)